Sample-to-Quote Integration: AI Usage Notes

Client	Perennials Fabrics (Bertram Capital)	**Date**	October 2026
Prepared by	Bennett Davis	**Repository**	github.com/bennettdavisv2/perennials-sample-to-quote

Which tools I used

Tool	Use
Claude Code (Claude Opus 5.5)	Coding agent that wrote the code, tests, demo and documentation under my direction.
Claude in Chrome	Clicked through the running demo to check layout and behavior.
Claude API (in the product)	Reads free-text quote emails into structured requests: SKU, quantity, unit and intent.

Where they helped

- **Speed to a working system.** A tested proof of concept, a multi-page demo and engineering specs within the engagement, with 54 automated tests.
- **Test coverage.** Turned the 21 supplied requests into test cases and designed 6 more edge cases. Two of them exposed real defects, both fixed.
- **Systems knowledge.** Raised production details early: Salesforce quote lines need each item in a price book, a pending NetSuite order does not hold inventory by default, and each customer's requests must be processed in order.
- **Reading email.** Maps informal wording ("the navy stripe") to catalog SKUs, constrained to a fixed schema, checked against the catalog and cross-checked by a rule-based parser.
- **Measured accuracy.** On all 13 workbook emails, hand-labeled, Claude reads every line exactly right (13/13 emails, 19/19 lines, about 3 seconds each). The rule-based parser gets 12/13, missing a quantity written as words; any disagreement goes to a representative. Reproducible with one command (python -m s2q.eval).

Where I had to correct them

- **It modelled the brief, not the product.** It parsed website requests as free text. I pointed it at the live checkout, where customers select exact colorways, and website intake became structured.
- **Its AI path had never run.** Current models rejected the API call it used, and the app silently fell back to rules. I had it switch to structured outputs, surface failures in the UI, and build a labeled evaluation so accuracy is measured rather than assumed.
- **It automated a judgment call.** It qualified new senders from what they wrote about themselves. I made qualification a representative's decision, because trade status controls free samples and pricing.
- **It over-corrected on ambiguity.** It held quotes to confirm colors that only one catalog item could match, recreating the back-and-forth the project removes. The final rule: quote a unique match, settle ties from the customer's sample history, and only then ask one question.
- **Its first interface was confusing.** I asked for a rebuild around a single request at a time, and removed a screen that added no value.